

Strategie di riduzione

È un algoritmo che quando abbiamo espressioni in cui sono presenti sottoespressioni "riducibili o trasformabili" sceglie quelle da trasformare "ridurre"

Quello che distingue un linguaggio funzionale da un altro sono soprattutto le strategie di riduzione

Haskell utilizza una strategia chiamata lazy, che riduce il sottotermine la cui riduzione è indispensabile per arrivare al risultato

Strategie di riduzione che non terminano

(alwaysresen (mf 3))

alwaysresen m = 4

mf m = mf m

se riduzione mf 3 = mf 3 = mf 3

se riduzione alwaysresen → 4

} due strategie

Funzione di ordine superiore

sono funzioni che prendono altre funzioni come argomento o che restituiscono funzioni come risultato

Esempio

ot zero f = f 0

ot zero prende come argomento una funzione e restituisce il valore di quest'ultimo sul numero 0

Haskell valuta l'espressione (ot zero square):

→ A sostituisce un nome con l'espressione associata ad esso

→ B sostituisce il parametro attuale al posto del parametro formale

formale

→ c Calcolo di una funzione predefinita

$$\rightarrow A ((\lambda f \rightarrow (f \circ)) \text{square})$$

$$\rightarrow B (\text{square } 0)$$

$$\rightarrow A ((\lambda m \rightarrow (m \cdot m) \circ)$$

$$\rightarrow B (0 \cdot 0)$$

$$\rightarrow c \ 0$$

} funzione m
formato haskell

Possiamo definire delle funzioni in cui gli argomenti e i risultati sono funzioni.

Esempio COMPOSE: che prende due funzioni (numerici) come argomenti e restituisce la loro composizione

$$\text{compose } ff = \lambda x \rightarrow (f(fx))$$

definiamo anche $me \ m = m + 1$

e poi valutiamo $((\text{compose square } me) \ 3)$

$$me \ 3 = 4$$

$$\text{square } 4 = 16$$

$$f(f(x)) \quad \begin{array}{c} \text{3} \\ \parallel \\ \text{me} \end{array} \quad \downarrow \text{me } 3 = 4$$

$$f(4) = 16 \quad \downarrow \text{square}$$

Possiamo dire $\text{square } me = (\text{compose square } me)$

CURRYFICAZIONE

Le nozioni di funzione di ordine superiore ci permette di introdurre quelle di "curryficazione"

Curryficare significa trasformare una funzione $f: (A_1 \cdot \dots \cdot A_n) \rightarrow B$

in funzioni di ordine superiore, prendono funzioni e restituiscono funzioni

$$f_c: A_1 \rightarrow (A_2 \rightarrow (\dots \rightarrow A_n \rightarrow B) \dots))$$

tale che

$$f(a_1, a_2, \dots, a_n) = f_c(a_1)(a_2) \dots (a_n)$$

Nel linguaggio Haskell le funzioni vengono implicitamente curryficate

Posiamo scrivere `compose` così:

$$\text{compose } f \ x = (f \ (f \ x))$$

Si può notare da una funzione curryficate od una non e siccome come conseguenze delle definizioni

Esempio:

nel λ -calcolo le funzioni che si introducono hanno un solo argomento e anche esse possono essere curryficate

Induzione nei programmi

L'uso dell'induzione per dimostrare proprietà di programmi permette l'uso di tecniche matematiche per manipolare programmi, per ottimizzarli oppure per dimostrare proprietà ~

La ricorsione è alla base della potenza computazionale nei linguaggi funzionali

In un programma ricorsivo l'output è definito inizialmente per gli elementi di base e poi è definito l'input considerando generico e assumendo che valga per i numeri più piccoli.

Il principio di induzione è una regola logica che implicitamente rappresenta anche un metodo di calcolo nelle prop. funzionali

Inoltre nelle prop. funz. corrisponde alle definizioni di funzioni numeriche per ricorsione come il "fact" di Haskell

Vogliamo dimostrare che per ogni input n il calcolo (le valutazioni) di $(\text{fact } n)$ terminano

Vogliamo mostrare:

1. le valutazioni $\text{fact } 0$ terminano
2. per un generico $k > 0$ le valutazioni di $\text{fact } k$ terminano utilizzando l'ipotesi induttiva $(\text{fact } (k-1))$ termini

Dimostrazione

1. $\text{fact } 0$ $\quad \quad \quad m \rightarrow \text{if } (m == 0) \text{ then } 1$
 $\quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \text{else } m \cdot (\text{fact } (m-1))$

2. Se $k > 0$

valutiamo $(\text{fact } k) \rightarrow k \cdot (\text{fact } (k-1))$

Per ipotesi induttiva $\text{fact } (k-1)$ è vero perché è

l'insieme di moltiplicazioni e sottrazioni che è
procedura termine

Anche per λ termine perché è una moltiplicazione
di qualcosa con qualcosa che termine

λ - calcolo

È un modello computazionale sul quale si basa il
linguaggio funzionale. È un tipo di linguaggio semplice
chiamato procedura

Basato su

Variabili $(x, y, z \dots)$

Astrazione o funzione anonima $\lambda x.M$ dove M è un
termine e x una
variabile

Applicazione: MN dove M ed N sono termini

Tutti questi rappresentano i λ -termini che sono
i programmi e i dati del modello

λ -termini costituiscono il λ -calcolo

Il λ -calcolo è definito dalle seguenti primitive

$$\Delta ::= x \mid (\Delta \Delta) \mid \lambda x. \Delta$$

Δ insieme dei λ -termini

x metavariable e appartiene all'insieme numerabile
di tutte le variabili

Astrazioni servono per creare funzioni anonime
(senza nome) $\lambda x.M$ "presa l'input x ritorna del
corpo di M "

Abbreviazioni:

$$(\lambda x_1 \cdot (\lambda x_2 \cdot \dots \cdot (\lambda x_n \cdot M))) \rightarrow (\lambda x_1 x_2 x_3 \dots x_n M)$$

$$(\dots (M_1 M_2) M_3) \dots M_n \rightarrow (M_1 M_2 M_3 \dots M_n)$$

Per esempio

$$\lambda x. (x (\lambda y. y x))$$

$$\lambda x. x (\lambda y. y x)$$

$\lambda x P$ P ambiente dell'ablogione

Nel termine, una variabile può essere limitata o rappresentare l'argomento del corpo della funzione, libera altrimenti.

variabile limitata o
corpo della funzione
(non locale)

Definizione variabile limitata

Definiamo $BV(M)$ l'insieme delle variabili limitate di M

Introducendo

$$BV(x) = \emptyset \text{ insieme vuoto}$$

$$BV(PQ) = BV(P) \cup BV(Q)$$

$$BV(\lambda x. P) = \{x\} \cup BV(P)$$

non si trova nella
definizione della funzione
ed libera
(non globale)

Definizione variabile libera

Definiamo $FV(M)$ l'insieme delle variabili libere di M

Introducendo

$$FV(x) = \{x\}$$

$$FV(PQ) = FV(P) \cup FV(Q)$$

$$FV(\lambda x. P) = FV(P) \setminus \{x\}$$

λ
è una variabile limitata

Sostituzione

Notazione: $M[L \setminus x]$ termine dove ogni variabile libera x , che non rappresenta un argomento della funzione, viene rimpiazzato da L

" $M[L \setminus x]$ significa sostituire tutte le occorrenze libere delle variabili x in $\underline{M}$ con l'espressione L ."
Lo corpo della funzione

Esempio:

$$M = x + 1$$

$$L = 5$$

x = variabile da sostituire

$$M[L \setminus x] = (x + 1)[5 \setminus x] = 5 + 1$$

Definizione di sostituzione

1) Se M è una variabile ($M=y$) allora

$$y[L/x] = \begin{cases} L & \text{se } x=y \\ y & \text{se } x \neq y \end{cases}$$

2) Se M è un'applicazione $M=PQ$ allora

$$PQ[L/x] = P[L/x] Q[L/x]$$

3) Se M è una λ -astrazione ($M=\lambda y.P$) allora

$$\lambda y.P[L/x] = \begin{cases} \lambda y.P & \text{se } x=y \\ \lambda y.(P[L/x]) & \text{se } x \neq y \end{cases}$$

sono
definizioni

Note bene

Si possono sostituire solo se le variabili che si sostituisce sono libere

Controesempio

in cui le variabili non sono libere

Funzioni costanti che ritornano sempre 2 per ogni argomento

$$\lambda x. 2 \quad \text{e} \quad \lambda y. 2$$

Sostituiamo x e z :

$$(\lambda x. 2)[x/z] \rightarrow \lambda x. 2[x/z] \rightarrow \lambda x. x \quad \text{funz identit }$$

$$(\lambda y. 2)[x/z] \rightarrow \lambda y. 2[x/z] \rightarrow \lambda y. x \quad \text{funz che ritorna sempre } x$$

Abbiamo ottenuto 2 funzioni diverse dopo la sostituzione